

Finishing the SV-16 board in KiCad — the exact tutorial

This is the click-by-click version of `hardware/README.md`. It assumes you have the repository checkout with `hardware/sv16_board/` in it and that you have never opened this particular board before. Everything below is specific to **this** board: the numbers, the net names, the part references and the order of operations are all the SV-16's, not generic advice.

What you start with. `hardware/sv16_board/`
`sv16_board.kicad_pcb` — a 4-layer, 100 × 100 mm board with 141 footprints placed, the complete netlist (472 pads on 133 nets), five net classes, **five zones** (solid GND on `In1.Cu`, a GND pour on `B.Cu`, and the 3V3 / 1V1 / 2V5 pours on `In2.Cu`), **291 vias and 163 stub tracks** (the escape vias on every pad that can legally drop into its own pour, plus GND stitching round the edge), and **no routed signal copper**. Plus `sv16_board.kicad_pro`, the project file that carries the design rules.

Companion files, all generated from the same data (make `pcb` rebuilds the lot):

File	Use it for
<code>hardware/sv16_board/pcb_top_view.png</code> , <code>pcb_bottom_view.png</code>	the board as it will look, before you open KiCad
<code>hardware/sv16_board/pcb_net_map.png</code>	the ratsnest by function — the routing plan on one page
<code>hardware/sv16_board/pcb_power_map.png</code>	the power tree and the <code>In2.Cu</code> pour map
<code>hardware/sv16_board/pcb_connection_sheets.png</code>	the six connection sheets, drawn
<code>hardware/sv16_board/BOM.csv</code> , <code>JLCPCB_BOM.csv</code> , <code>JLCPCB_CPL.csv</code> , <code>FAB_NOTES.md</code>	ordering and assembly
<code>PCB_CONNECTIONS.md</code>	the wiring part by part (§3) and the checks before routing (§9)

What "done" means. Every net routed or deliberately left unrouted, DRC clean apart from cosmetic silkscreen warnings, and a gerber set you can upload. Budget two days: one for placement review + power + the FPGA fan-out, one for signals, DRC and outputs. Four-layer, 141 parts — that is realistic for someone who knows KiCad, and tight for someone who doesn't.

0. Install and open

1. Install **KiCad 7.0 or newer** (`kicad.org` → Download). The file is saved in KiCad 7's format (`version 20221018`); KiCad 8, 9 and later will offer to migrate it on first save — accept, the geometry and nets are unchanged.
2. Copy the whole `hardware/sv16_board/` folder somewhere you own — the `.kicad_pcb`, the `.kicad_pro`, and `placement_preview.png` belong together. Never edit the copy in the repo: make `kicad-board` regenerates it and will overwrite your work.
3. Double-click `sv16_board.kicad_pro`. KiCad opens the PCB editor with the project's net classes loaded. (Opening the `.kicad_pcb` alone also works, but then the net classes are whatever KiCad last used — open the project.)
4. If KiCad asks about a footprint library table or "some libraries are missing", ignore it for now — every footprint this board references exists in the official KiCad libraries (they were checked against the KiCad footprint repository), and the next step fixes the rest.

Sanity check before you touch anything — if any of these is false, stop and re-clone:

Check	Where	Expected
Board outline	Edge.Cuts tab	a closed 100 × 100 mm square, 4 mounting holes, 4 fiducials
Footprint count	status bar / Inspect → Net Inspector	141 footprints
Nets	Inspect → Net Inspector	133 nets; the list starts GND, VM_IN_RAW, VM_IN, 3V3, 2V5, 1V1 ...
Ratsnest	View → Show Ratsnest (or the toolbar)	a spider's web — that is normal, nothing is routed yet
Zones	View → Show Zone Fills	five zones: GND on In1.Cu and B.Cu, 3V3/2V5/1V1 on In2.Cu — unfilled until you press B
Copper	View → Show Ratsnest + the layer tabs	291 vias and 163 short stubs on F.Cu; no routed signals yet

1. The one mandatory step: update the footprints from the library

The board file carries **generated** pads: correct pad numbers, correct positions and correct nets, but approximate pad sizes and no courtyard/3D data. Replace them with the real library footprints before you route:

1. **Tools → Update Footprints from Library...**
2. In the dialog: tick "**Update footprints**", leave "**Change footprint library links**" on Always (the library IDs are already right), leave the rest at default, and press **Update**.
3. KiCad reports how many footprints were updated. Every placement and every net is preserved — this is a pad swap, nothing moves.

Do this **now**, not later: routing to the wrong pad size and then changing the footprint moves the already-routed copper.

If a footprint is reported missing, it is a library, not a board, problem: **Preferences → Manage Footprint Libraries** → make sure the standard KiCad library (the one that ships with KiCad) is in the table and enabled.

If you would rather use your own part instead of a library footprint (a different inductor, a different oscillator can), do it here: select the footprint, press **E** to edit its properties, and change the library link to the part you bought.
hardware/cart/
RECONCILED.md lists which footprints the bought parts need.

2. Ten minutes of orientation (worth it)

- **Layer tabs** on the right: F.Cu, In1.Cu, In2.Cu, B.Cu, plus mask/silk. The stack is **signal / solid GND / solid power / signal** — say it to yourself once, because it decides the whole routing strategy: most signals go on the top layer, the ground plane is one Via below, and the rails live on In2.Cu as pours.
- **Inspect → Net Inspector**: sort by "Unrouted" — this is your scoreboard for the next two days.
- **Alt+3**: the 3D viewer. Look at the connectors — J8 (USB-C) must sit flush with the edge, J9 (barrel) wants its opening at the edge, J3/J4/J7 are 2.54 mm headers that must not collide.
- **Ctrl+S** often. KiCad does not autosave to disk the way an editor does.

3. Fix the floorplan before you route anything

The placement in the file is a real floorplan, not a dump: power top-left, FPGA centre, both flashes right of it, USB and console bottom-right, JTAG and expansion along the bottom/left, LEDs along the bottom edge. Moving a part before routing costs seconds; after routing it costs an hour.

Check these, in this order (all of them come from BOARD.md §11):

What	Where it should be	Why
Y1 (25 MHz XO)	as close as you can get to U1 pin 133 , on the top layer. The shipped placement has it ≈18 mm away in the upper-left cluster — workable at 25 MHz, better if you pull it in	that is <code>clk_25m</code> ; a long clock trace picks up the switchers and makes bring-up guesswork
U5 / U6 switchers, L1/L2, C38-C41	tight cluster, away from Y1, U2, U4	switching edges couple into a crystal oscillator and into anything analog
U2 (configuration flash)	within ~15 mm of U1 pins 54/46/47/49 (CCLK, MISO, MOSI, CSSIPIN)	these four are also the JTAG chain's neighbours, and long stubs hurt the boot
U4 (CH340G) + Y2/C36/C37	next to J8/U8, crystal traces short and symmetric	USB and a 12 MHz crystal in one corner, away from the switchers
U8 (USBLC6)	between J8 and U4	it protects U4; it must be on the connector side
J8	at the board edge	the plug shell hangs off the edge
TP1-TP6	reachable with a probe while the board is powered	3V3, 2V5, 1V1, GND, DONE, INITN

Tools you will use: **M** move, **R** rotate (45° steps with the mouse, 90° with a click), **F** flip to the other side, **E** properties (to type an exact X/Y), **Ctrl+M** move exactly.

The placement is already checked for collisions. The generator runs a repair pass over every part: a deterministic relaxation that nudges a part until no pad of one net comes within 0.2 mm of a pad of another, no more than 9 mm from where the floorplan table put it (`effective_placement()` in `scripts/sv16_board_kicad.py`). It moved 97 parts on the first run — the hand-typed table had overlapping pads, which is a short before the board is even routed. If you move a part, run `python3 scripts/sv16_board_kicad.py --check`: it re-scans every pad pair and prints any conflict it finds.

Two placement rules that are not obvious:

- Put **C38** (100 nF, the AP62300 bootstrap) right next to U5's BST and SW pins. Together with the SW trace it forms the highest-dV/dt loop on the board.
- Put **C39/C40/C41** tight around U5, and take the R46/R47 feedback tap **at the output capacitor's pad**, not at the far end of the copper. That is what keeps the 3.3 V rail quiet.

4. Set the design rules (5 minutes, do not skip)

File → Board Setup → Design Rules → Net Classes. The project already carries these — verify them, because everything you route will inherit them:

Net class	Nets	Track	Clearance	Via
Default	everything else	0.25 mm	0.20 mm	0.8 / 0.4 mm
Power	GND, VM_IN, VM_IN_RAW, 3V3, 2V5, 1V1, USB_VBUS, 5V_USB	0.8 mm	0.25 mm	1.0 / 0.5 mm
Switching	U5_SW, U5_BST, U6_SW, U6_BST	0.8 mm	0.50 mm	1.0 / 0.5 mm
USB	USB_DP, USB_DN, USB_DP_F, USB_DN_F, CC1, CC2	0.25 mm	0.20 mm	0.8 / 0.4 mm
JTAG	TCK, TMS, TDI, TDO, PROGRAMN, INITN, DONE, CCLK	0.25 mm	0.25 mm	0.8 / 0.4 mm

Then, in the same dialog:

- **Constraints:** minimum clearance 0.2 mm, minimum track width 0.2 mm, minimum via 0.5 mm, minimum annular ring 0.13 mm, minimum through-hole 0.3 mm. (The project sets these already.)
- **Track & Via Dimensions:** add a third via size **0.45 / 0.2 mm** — you will need it for the FPGA fan-out and nowhere else.
- **Differential pair** (for the USB class): width **0.20 mm**, gap **0.20 mm**. On the standard 4-layer stack that lands in the 85–95 Ω range. If you want it exact, turn on your fab's impedance control and let them confirm the pair geometry; for USB full-speed (12 Mbit/s) anything in that range works fine — this is not a 5 Gbit/s link.

Do **not** raise the Default class to 0.25 mm clearance "to be safe". 0.5 mm-pitch TQFP fan-out with 0.2 mm traces needs every bit of the 0.2 mm clearance.

5. Fill the zones

The board ships with **five zones defined but unfilled**: a solid GND plane on **In1.Cu**, a GND pour on **B.Cu**, and the three rail pours on **In2.Cu** (3V3 across the board at priority 0, with 1V1 and 2V5 carved out of it where their regulators, bulk capacitors and decoupling sit).

1. Press **B** (Edit → Fill All Zones). The zones fill around every pad and track.
2. Look at In1.Cu; the plane should cover the whole board. Then check for **isolated islands** — small copper patches with no path to a GND pad: *Inspect* → *Design Rules Checker* → run DRC; it reports them as isolated copper warnings. Fix each one by moving the offending parts/tracks, or by deleting the island (select it, **Del**), or by adding a GND stitching via into it.
3. Sanity: with the GND zone visible, follow the plane from the FPGA to U1's ground pins. If the plane is cut in two anywhere by a row of tracks, that is a design bug — fix the tracks, not the zone.

Fine tuning: if the fill can't reach into a tight pocket (under U1, between the flash pads), lower that zone's clearance to 0.2 mm in its properties (**E** with the zone selected → Clearance).

6. Check the **In2.Cu** pours (they are already drawn)

In2.Cu carries three zones, drawn by `scripts/sv16_pcb_copper.py` and verified against every pad on the board:

Zone	Priority	Covers
3V3	0	the whole board
2V5	2	the LP5907 group, plus the band above U1 where C8–C11 sit
1V1	3	the MP1584 group, L2, C21, D11, the feedback divider, plus the strip down the left of U1 that feeds C2–C7

They **may** overlap — KiCad fills the highest priority first and cuts the lower one back, so there is no "copper zone overlap" error to chase. What you should do instead:

1. Press **B** and look at In2.Cu with the three nets set to different colours (Appearance panel): every GND pad should have a via into In1.Cu, every rail pad should sit inside its own pour.
2. Seven pads are deliberately **not** connected to the pours by a via, because they sit over the wrong rail's copper — U1 pins 17, 20, 38, 66, 83, 96, 130 and 132. Route each with a 0.25 mm track to its own patch. The list is printed by `python3 scripts/sv16_pcb_copper.py --report` and in `PCB_CONNECTIONS.md` §7.
3. If you move U5, U6, U7 or their capacitors, move the pour boundary with them — or edit the polygons in `sv16_pcb_copper.py` and re-run `make pcb`.

Why pours and not traces: a 1.1 V rail at ~400 mA through a 0.8 mm trace from one corner of a 100 mm board drops tens of millivolts and adds inductance right where the FPGA wants none. The pour makes the drop negligible and decouples the rail for free.

7. Route — in this order, with these widths

Enable the interactive router: **Route** → **Interactive Router Settings** → **Shove** (it pushes existing copper out of the way instead of letting you create a short). Core shortcuts: **X** route a track, **V** drop a via while routing, **D** drag, **6** route a differential pair, **Del** delete.

7.1 Power first (1-2 h)

Route **VM_IN** from J9 through D8/FB1 to U5 and U6 at **0.8 mm**, with **GND** returns beside it. Then the rails: **3V3** out of L1, **1V1** out of L2, **2V5** out of U7. Every rail gets its local capacitors soldered to the same copper — route through the cap pads, not past them.

7.2 The switcher loops (30 min, and this is where boards fail)

For **U5**: **VIN** cap → U5 pin 3 → U5 pin 2 (**SW**) → L1 → output caps, all on the top layer, all short. C38 sits across **BST** (pin 6) and **SW** (pin 2). The feedback divider (R46/R47) taps the **3V3** node **at C40/C41**, and the **FB** trace stays away from the **SW** node. For **U6** the same, plus the **D11** catch diode: its cathode goes to **U6_SW**, anode to **GND**, with the shortest possible return.

7.3 The FPGA fan-out (2-3 h)

U1 is TQFP-144, 0.5 mm pitch. Make a fan-out "comb": from every pad, a short **0.2 mm** track straight out, then a **0.45 / 0.2 mm via** into the layer that suits the net — signals to **B.Cu**, power to **In2.Cu**, ground straight into the **In1.Cu** plane. Do it pin-by-pin; there is no trick, and doing it first means the inner layers stay free for the pours.

The GND and supply pins are already done. The generator placed a 0.45/0.2 mm via with a 0.2 mm stub on every surface-mount pad that sits over its own rail's pour — all 14 **GND** pads of U1 straight into the **In1.Cu** plane, the nine **3V3** pads into the **In2.Cu** **3V3** pour, and the **1V1/2V5** pads that are over their own patches. That is 291 vias of the fan-out already on the board; what is left for you is the **signal** comb (the 52 constrained I/O plus the spare I/O pins), which goes to **B.Cu** through 0.45/0.2 mm vias.

Keep the three clock/config pins (54 **CCLK**, 133 **clk_25m**, 57 **PROGRAMN**) short and away from the switchers.

7.4 USB (45 min)

USB_DP / USB_DN (and **USB_DP_F / USB_DN_F** after U8) are a differential pair. Route J8 → U8 → U4 with **6** (Route Differential Pair): 0.2 mm tracks, 0.2 mm gap, both on the top layer, **no vias**, side by side the whole way, length difference under 0.5 mm, and no stubs. Put a **GND** trace or the bottom-layer pour alongside them; the ground plane under the pair carries the return. This is the one place where the layout decides whether USB enumerates on the first try.

7.5 Everything else (2-3 h)

Flash SPI (U2/U3), the CH340G UART pairs, LEDs and their resistors, JTAG, expansion headers, motor connector. Let the ratsnest guide you; it knows every connection. Net names are self-documenting — **flash_cs_n**, **led[2]**, **spi0_mosi**.

8. Second pass

- **Stitching vias**: 128 are already placed in two rings round the board edge (2.5 mm and 7.0 mm in, 5 mm apart, every one checked for clearance). Add more either side of the USB pair after you route it — a short **GND** track, press **V**, continue. This ties the top pour, the inner plane and the bottom pour into one ground, which is the entire point of a 4-layer board.

- **Thermals:** check that U5, U6 and U7 have copper to spread heat, and that the ground pads are connected with four spokes, not one.
- **Silkscreen:** after routing, move reference designators off pads and tracks. No reference text on a pad (it gets printed on the solder and DRC will warn). Legends for the connectors already exist on the silkscreen.

9. DRC — run it, then fix it in this order

Inspect → Design Rules Checker → Run DRC (with Refill all zones before performing DRC ticked). Expect these and deal with them as follows:

Message	What it means	Fix
Unconnected items	the scoreboard for this step	route it, or accept it deliberately (see below). Seven of them (U1's 1V1/2V5 pins) are listed in §6 — they are the pads that must be routed to their pour by hand
Clearance violation	two different nets too close	use the router's shove to open a corridor; do not shrink the rule below 0.2 mm
Track has unconnected end	a stub you started and abandoned	delete it or finish it
Isolated copper	a plane island with no connection	add a stitching via, or delete the island
Courtyard overlap	two parts physically intersect	move one; the 3D viewer makes it obvious
Hole clearance / annular ring	a via too close to an edge or pad	move it, or use the 0.45/0.2 via
Silkscreen overlap	cosmetic: text over a pad	move the text; safe to ignore if you are in a hurry
Zone has no net	you drew a zone without assigning a net	E → pick 3V3/1V1/2V5

Deliberate exceptions are fine — a genuinely unused net (the spare pins 109/144, the unused U4 modem pins) can stay unrouted. Mark them as excluded in the DRC dialog so the report stays honest.

Rule of thumb: do not accept a single copper error to make DRC quiet. Every copper error is a spinner, a short, or a dead board.

10. Look at it in 3D (**Alt+3**)

- J8 (USB-C) flush with the edge, J9 (barrel) with its opening clear of anything.
- No part hanging over the board edge, no tall part where a cable has to bend.
- The mounting holes have clearance for an M3 screw head.
- Flip the board (**v** in the 3D viewer) and check the bottom silk.

11. Gerbers and drill files

File → Fabrication Outputs → Gerbers (.gbr) — or **File → Plot** in older versions.

Setting	Value
Output directory	a fresh empty folder, e.g. <code>gerbers/</code>
Layers	F.Cu, In1.Cu, In2.Cu, B.Cu, F.Mask, B.Mask, F.Silkscreen, B.Silkscreen, Edge.Cuts — plus F.Paste if you are ordering a stencil

Setting	Value
Format	Gerber, 4.6 unit mm, no X2 changes needed
Options	Subtract soldermask from silkscreen on (the default), Use Protel filename extensions off unless your fab asks

Then, still in the plot dialog, **Generate Drill Files**: Excellon, **mm**, PTH and NPTH in one file, decimal format, no mirroring, and generate. You should end up with 9–10 .gbr files, two .drl files or one merged, and a .gbrjob.

hardware/sv16_board/

FAB_NOTES.md has the same settings as a card, with the counts filled in from the board file (they cannot drift), plus the BOM and CPL columns your fab's assembly service wants.

Verify before uploading. Open the files in **Gerber Viewer** (KiCad's, [File → Open Gerber Files...](#)) and click through the layers: four copper layers, each with the right features; the silkscreen readable (not mirrored); the outline closed. If a layer is empty or unrecognisable, you exported the wrong thing — fix it here, not at the fab.

12. Order the board

Any 4-layer fab works. For JLCPCB (the cheapest route to India, 4-layer 100 × 100 mm):

1. Upload the **zip** of the gerber folder. JLCPCB renders it and shows you the layers — check them in their viewer, not just yours.
2. Options: **4 layers**, 1.6 mm, **1 oz outer / 0.5 oz inner** (their default 4-layer stack), HASL (lead-free), 5 pieces, no impedance control required for full-speed USB at this geometry.
3. The board is exactly 100 × 100 mm — that is within their standard 4-layer price tier, but it is the boundary, so if the quote looks wrong you have drifted into the next size class. Check the board outline is exactly 0,0 → 100,100 in [Board Setup → Board Finish](#) / the [Edge.Cuts](#) edges.
4. Add a **stencil** if you are assembling by hand: the QFP-144 at 0.5 mm pitch and the 0.5 mm-pitch QFNs are painful without paste. The [F.Paste](#) layer is the stencil layer.
5. Expect roughly ₹1,000–1,800 for five 4-layer boards landed in India, a few days to ship, and a week or two in transit. PCBWay and Aisler are the alternatives; the files are fab-agnostic.

13. When the boards arrive: assembly and first power-up

Solder in this order — do not skip the checks:

1. Solder the **power section only**: U5, U6, U7, L1/L2, D8/D9/D11, the caps, FB1, J9. Nothing else.
2. Power from a **current-limited bench supply at 12 V, limit 200 mA**, through J9.
3. Measure, before anything else, at the test points:

Test point	Rail	Expected	If it is wrong
TP1	3V3	3.28 V (R46/R47 divider)	check R46/R47, the BST cap C38, that EN (pin 5) sits at ~3 V — not 12 V
TP3	1V1	1.10 V	check R38/R39 and the MP1584 COMP network (R45/C35)
TP2	2V5	2.5 V , appearing ~20 ms after 3V3	check R43/C32 on the LP5907's EN pin
TP4	GND	0 V	—

Current draw with nothing else fitted should be a few tens of mA. If the limit trips, remove power, find the short with a multimeter before reapplying. 4. Only then fit everything else — U1 (the FPGA) **last** among the ICs, and

the flashes before it. 5. Power again, fitted, and measure the rails once more. A 3.3 V rail that sags when the FPGA is fitted means a short under the QFP or a missing decoupling cap. 6. Plug in USB: the CH340G should appear as a serial port (`dmesg` on Linux, Device Manager on Windows — the CH340 driver may need installing). That proves Y2, the crystal circuit, U8 and the pair are all correct. 7. Bring-up to a running program: `docs/BOOT_AND_PROGRAMMING.md` and `docs/SYNTHESIS_AND_DEPLOYMENT.md` have the JTAG and flash sequence (`openFPGALoader` / `ecpprog`). DONE (TP5) goes high when the FPGA has configured; INITN (TP6) is your failure indicator.

If DONE never rises: measure `clk_25m` at Y1 pin 3 with a scope (25 MHz square wave — if it is missing, Y1 is the wrong part type, see `hardware/cart/RECONCILED.md`), then re-check PROGRAMN (pin 57), CCLK (54) and the configuration flash wiring.

Appendix A — keyboard card

Key	Action	Key	Action
X	route a track	B	fill all zones
V	drop a via (while routing)	E	edit the selected item's properties
D	drag a track/part	M	move
6	route a differential pair	R / F	rotate / flip to the other side
Del	delete	Ctrl+M	move exactly (type coordinates)
Alt+3	3D viewer	Home	zoom to fit
Ctrl+S	save	Ctrl+Z	undo (your best friend)

Appendix B — the five things that go wrong, and what they look like

1. **"My footprints moved / lost their nets."** You edited the `.kicad_pcb` from the repo instead of your own copy, and `make kicad-board` regenerated it. Work on a copy.
2. **"Update Footprints from Library did nothing."** The dialog needs the Update footprints box ticked, and the footprints must have library links (they do).
3. **"DRC is full of clearance errors I didn't create."** You are routing at the Default class width through a 0.5 mm-pitch area. Use the Power class for rails, and 0.2 mm tracks around U1.
4. **"The zones disappeared."** Zone fills are not saved unless you save the board after filling: fill (**B**) then save (**Ctrl+S**). KiCad also silently refills on DRC if you ticked the box.
5. **"USB does not enumerate."** Almost always the pair: a via in the pair, a long stub, the pair split around a part, or the CC resistors missing (R36/R37, 5.1 k to GND — they tell the host this is a device).

Appendix C — still to buy before the board is buildable

From `PCB_COMPONENTS.md` §10 and `hardware/cart/RECONCILED.md` §5:

- **10 kΩ 1 %** resistor (R47) — the AP62300 feedback bottom leg.
- **Ceramics:** 10 μF (C39), 2 × 22 μF (C40, C41), 10 μF for 2V5 (C42), and 10 μF ×2 for C25/C30.
- **An active 25 MHz 3.3 V XO** for Y1 — a 2-pin HC49/US crystal cannot clock the FPGA.
- Your call: the 470 μF input bulk in the cart is a **16 V** part on a 12 V rail (1.33× derating).

Everything else on the board is in the cart. `PCB_CONNECTIONS.md` §11 keeps the same list, and the DNP column of `hardware/sv16_board/BOM.csv` says which parts are deliberately not fitted (D6, J7).

Appendix D — the generator commands

Command	Does
<code>make pcb</code>	board file + bitmaps + BOM/CPL + FAB_NOTES.md + this project's PCB_CONNECTIONS.md
<code>make pcb-check</code>	parse the board back: counts, nets on existing pads, DRC-lite on 291 vias and 163 stubs, pad-to-pad collision scan
<code>python3 scripts/ sv16_pcb_copper.py --report</code>	what the copper plan connected, and the pads the router still owes
<code>python3 scripts/ sv16_pcb_bitmap.py --scale 40</code>	bigger bitmaps (40 px/mm instead of 24)
<code>make pcb-connections-pdf</code>	PCB_CONNECTIONS.pdf for printing

Regenerate this tutorial's companion artifacts with `make kicad-board` (board + preview) and `make board-pdf / make pcb-doc` (the two reference PDFs). The generator's self-check is `python3 scripts/
sv16_board_kicad.py --check`.